

## **Exp: 8 Write the Python Program to Implement DFS.**

### **Aim:**

To write a python program to implement DFS.

### **Objective :**

To implement the **Depth First Search (DFS)** algorithm in Python for traversing all nodes of a graph from a given starting vertex.

### **Algorithm:**

**Step 1:** Initialize a set to keep track of visited nodes.

**Step 2:** Call the DFS-Visit function with the starting node and the set of visited nodes.

**Step 3:** In the DFS-Visit function, add the current node to the set of visited nodes.

**Step 4:** For each neighbor of the current node, if it has not been visited yet, recursively call the DFS-Visit function with the neighbor node and the set of visited nodes.

**Step 5:** Repeat steps 3-4 until all reachable nodes have been visited.

### **Program:**

```
# Define the graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Define the DFS function
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start) # mark the node as visited
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited) # recursively visit unvisited neighbors
    return visited

# Call the DFS function with starting node 'A'
print(dfs(graph, 'A'))
```

### **Output:**

```
{'A', 'B', 'C', 'E', 'F', 'D'}
```

```
Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/madhulikavanjavaka/Desktop/AI/Exp 8.py =====
{'A', 'B', 'C', 'E', 'F', 'D'}
>>>
```

**Result:**

Thus, the Program was executed and output found to be correct.

## Exp: 9 Write the Python to Implement Travelling Salesman Problem.

### Aim:

To write a python program to implement travelling salesman problem.

### Objective:

To implement the **Travelling Salesman Problem (TSP)** in Python to determine the shortest possible route that visits each city exactly once and returns to the starting city.

### Algorithm:

**Step 1:** Initialize the best distance and best route to be infinity and an empty list, respectively.

**Step 2:** Generate every possible permutation of the cities.

**Step 3:** For each permutation, calculate the distance of the path by summing the distances between each pair of consecutive cities in the permutation.

**Step 4:** Add the distance from the last city to the first city.

**Step 5:** If the calculated distance is shorter than the current best distance, update the best distance and best route.

**Step 6:** Return the best distance and best route.

### Program:

```
# Graph represented as an adjacency list
```

```
graph = {  
    'A': ['B', 'F'],  
    'B': ['C', 'D'],  
    'C': [],  
    'D': ['E'],  
    'E': [],  
    'F': ['G'],  
    'G': ['H', 'I'],  
    'H': [],  
    'I': ['J'],  
    'J': []  
}
```

```
# DFS function to find a path
```

```
def dfs(graph, start, goal, path=None):
```

```
    if path is None:
```

```
        path = []
```

```
    path = path + [start]
```

```

    if start == goal:
        return path
for node in graph[start]:
    if node not in path:
        new_path = dfs(graph, node, goal, path)
        if new_path:
            return new_path
return None

# Driver code
path = dfs(graph, 'A', 'J')

if path:
    print("Path found:", path)
else:
    print("No path found.")

```

### Output:

Path found: ['A', 'F', 'G', 'T', 'J']



```

Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-
2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.
>
==== RESTART: /Users/madhulikavanjavaka/Desktop/AI/exp 9.py =====
Path found: ['A', 'F', 'G', 'I', 'J']
>

```

### Result:

Thus the Program was executed and the output was found to be Correct.

## **Exp: 10 Write the python program to implement A\* algorithm.**

### **Aim:**

To write a python program to implement A\* algorithm.

### **Objective:**

To implement the **A\* (A-Star)** algorithm in Python to find the shortest path between a start node and a goal node using heuristic search.

### **Algorithm:**

**Step 1:** Initialize the open list with the starting node and the closed list as an empty set.

**Step 2:** Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.

**Step 3:** While the open list is not empty, select the node with the lowest f score and set it as the current node.

**Step 4:** If the current node is the goal node, return the reconstructed path. Remove the current node from the open list and add it to the closed list.

**Step 5:** For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.

**Step 6:** If the neighbor is already in the closed list, skip it.

**Step 7:** If the neighbor is not in the open list, add it and set its g score and f score.

**Step 8:** If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.

**Step 9:** Otherwise, update the came-from dictionary, g score, and f score of the neighbor.

**Step 10:** Repeat steps 3-10 until the open list is empty. If the goal node was never reached, return failure.

### **Program:**

```
import heapq
```

```
# 5x5 Grid (0 = free path, 1 = obstacle)
```

```
grid = [  
    [0, 0, 0, 0, 0],  
    [1, 1, 0, 1, 0],  
    [0, 0, 0, 1, 0],  
    [0, 1, 1, 0, 0],  
    [0, 0, 0, 0, 0]  
]
```

```
# Find neighboring cells
```

```
def grid_neighbors(node):
```

```

x, y = node
neighbors = []

directions = [(0,1), (1,0), (0,-1), (-1,0)]

for dx, dy in directions:
    nx, ny = x + dx, y + dy

    if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]):
        if grid[nx][ny] == 0:
            neighbors.append((nx, ny))

return neighbors

# Manhattan Distance Heuristic
def grid_heuristic(node, goal):
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])

# A* Algorithm
def astar(start, goal, neighbors_fn, heuristic_fn):

    frontier = []
    heapq.heappush(frontier, (0, start))

    came_from = {}
    cost_so_far = {}

    came_from[start] = None
    cost_so_far[start] = 0

    while frontier:

```

```

_, current = heapq.heappop(frontier)

if current == goal:
    break

for neighbor in neighbors_fn(current):

    new_cost = cost_so_far[current] + 1

    if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:

        cost_so_far[neighbor] = new_cost

        priority = new_cost + heuristic_fn(neighbor, goal)

        heapq.heappush(frontier, (priority, neighbor))

        came_from[neighbor] = current

# Reconstruct path
path = []
current = goal

while current is not None:
    path.append(current)
    current = came_from[current]

path.reverse()

return path, cost_so_far[goal]

# Driver Code

```

```
start = (0, 0)
```

```
goal = (4, 4)
```

```
path, cost = astar(start, goal, grid_neighbors, grid_heuristic)
```

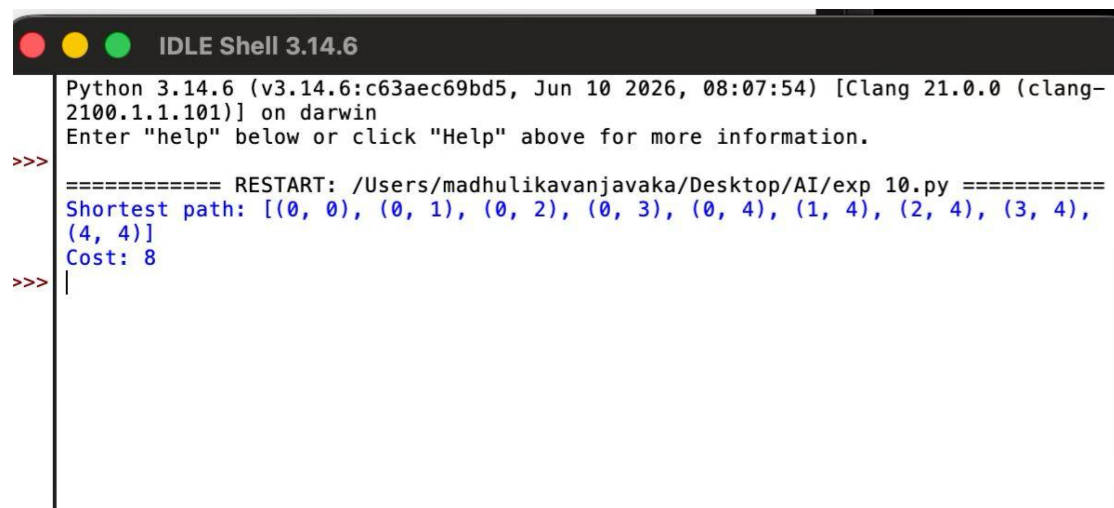
```
print("Shortest path:", path)
```

```
print("Cost:", cost)
```

### Output:

Shortest path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]

Cost: 8



```
Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: /Users/madhulikavanjavaka/Desktop/AI/exp 10.py =====
Shortest path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
Cost: 8
>>> |
```

### Result:

Thus the Program was executed and the output was found to be Correct.



## Exp: 11 Write the python program for Map Coloring to implement CSP.

### Aim:

To write a python program for map coloring to implement CSP.

### Objective:

To implement the **Map Coloring** problem using **Constraint Satisfaction Problem (CSP)** techniques in Python, ensuring that no two adjacent regions are assigned the same color.

### Algorithm:

**Step 1:** Initialize an empty assignment for the map coloring problem.

**Step 2:** For each country in the map, add a variable  $X_i$  to the assignment.

**Step 3:** Define the constraints as pairs of adjacent countries, and add a constraint  $(X_i, X_j)$  to the CSP such that  $X_i \neq X_j$ .

**Step 4:** Define the domain of each variable to be the set of available colors.

**Step 5:** Call the Backtrack function to find a consistent assignment for the variables.

**Step 6:** In the Backtrack function, check if the assignment is complete. If it is, return the assignment.

**Step 7:** Select an unassigned variable  $X_i$  from the assignment.

**Step 8:** For each value  $v$  in the domain of  $X_i$ , check if it is consistent with the constraints and the assignment so far.

**Step 9:** If it is, assign  $X_i = v$  to the assignment and recursively call Backtrack with the new assignment. If the result is not a failure, return the result.

**Step 10:** If the result is a failure, remove the assignment  $X_i = v$  and try the next value in the domain. If all values in the domain have been tried and failed, return failure.

### Program:

```
from typing import Dict, List
from itertools import product
from copy import deepcopy
```

```
class CSP:
```

```
    def __init__(self, variables: List[str], domains: Dict[str, List[str]], constraints):
        self.variables = variables
        self.domains = domains
        self.constraints = constraints
```

```
    def solve(self):
        assignments = {}
        return self.backtrack(assignments)
```

```
    def backtrack(self, assignments):
        if len(assignments) == len(self.variables):
            return assignments
        var = self.select_unassigned_variable(assignments)
        for value in self.order_domain_values(var, assignments):
            new_assignments = deepcopy(assignments)
            new_assignments[var] = value
            if self.is_consistent(new_assignments):
                result = self.backtrack(new_assignments)
                if result is not None:
```

```

        return result
    return None

def select_unassigned_variable(self, assignments):
    for var in self.variables:
        if var not in assignments:
            return var

def order_domain_values(self, var, assignments):
    values = self.domains[var]
    return sorted(values, key=lambda value: self.get_num_conflicts(var, value, assignments))

def get_num_conflicts(self, var, value, assignments):
    conflicts = 0
    for constraint in self.constraints:
        if var in constraint and len(constraint) == 2:
            other_var = constraint[0] if constraint[1] == var else constraint[1]
            if other_var in assignments and assignments[other_var] == value:
                conflicts += 1
    return conflicts

def is_consistent(self, assignments):
    for constraint in self.constraints:
        if all(var in assignments for var in constraint):
            if not self.satisfies_constraint(assignments, constraint):
                return False
    return True

def satisfies_constraint(self, assignments, constraint):
    if len(constraint) == 2:
        val1, val2 = assignments[constraint[0]], assignments[constraint[1]]
        return val1 != val2
    else:
        return True

def map_coloring():
    variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
    domains = {
        'WA': ['r', 'g', 'b'], 'NT': ['r', 'g', 'b'], 'SA': ['r', 'g', 'b'],
        'Q': ['r', 'g', 'b'], 'NSW': ['r', 'g', 'b'], 'V': ['r', 'g', 'b'], 'T': ['r', 'g', 'b']
    }
    constraints = [
        ('WA', 'NT'), ('WA', 'SA'), ('NT', 'SA'), ('NT', 'Q'), ('SA', 'Q'),
        ('SA', 'NSW'), ('SA', 'V'), ('Q', 'NSW'), ('NSW', 'V')
    ]
    csp = CSP(variables, domains, constraints)
    result = csp.solve()
    if result is not None:
        print(result)
    else:
        print("No solution found")

map_coloring()

```

## Output:

```
{'WA': 'r', 'NT': 'g', 'SA': 'b', 'Q': 'r', 'NSW': 'g', 'V': 'r', 'T': 'r'}
```

```
IDLE Shell 3.14.6
Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/madhulikavanjavaka/Desktop/AI/exp 11.py =====
{'WA': 'r', 'NT': 'g', 'SA': 'b', 'Q': 'r', 'NSW': 'g', 'V': 'r', 'T': 'r'}
>>> |
```

**Result:**

Hence, the simple python program for Map Coloring to implement CSP is done.

## Exp: 12 Write the python program for Tic Tac Toe game.

### Aim:

To write a python program for Tic Tac Toe game.

### Objective:

To implement the **Tic Tac Toe** game in Python, allowing two players to play interactively while checking for valid moves, winning conditions, and a draw.

### Algorithm:

**Step 1:** Initialize the board as a list of 9 empty spaces.

**Step 2:** Print the current board in a 3x3 grid format.

**Step 3:** Prompt the current player to enter a move between 1 and 9 and place their icon if the chosen cell is empty.

**Step 4:** After every move, check all winning combinations (rows, columns, diagonals) to see if the current player has won.

**Step 5:** If no winner is found, check whether the board is full to declare a draw.

**Step 6:** Alternate turns between player X and player O and repeat steps 2-5 until a win or draw occurs.

### Program:

```
# Tic Tac Toe game
board = [' ' for _ in range(9)]

def print_board():
    row1 = ''.join(board[0:3])
    row2 = ''.join(board[3:6])
    row3 = ''.join(board[6:9])
    print(row1)
    print('-' * 5)
    print(row2)
    print('-' * 5)
    print(row3)

def player_move(icon):
    if icon == 'X':
        player = 1
    else:
        player = 2
    print("Player {}'s turn".format(player))
    choice = int(input("Enter your move (1-9): ").strip())
    if board[choice-1] == ' ':
        board[choice-1] = icon
    else:
        print("That space is already taken!")

def is_victory(icon):
    if (board[0] == icon and board[1] == icon and board[2] == icon) or \
       (board[3] == icon and board[4] == icon and board[5] == icon) or \
       (board[6] == icon and board[7] == icon and board[8] == icon) or \
       (board[0] == icon and board[3] == icon and board[6] == icon) or \
       (board[1] == icon and board[4] == icon and board[7] == icon) or \
       (board[2] == icon and board[5] == icon and board[8] == icon) or \
       (board[0] == icon and board[4] == icon and board[8] == icon) or \
       (board[2] == icon and board[4] == icon and board[6] == icon):
        return True
```

```

    else:
        return False

def is_draw():
    if ' ' not in board:
        return True
    else:
        return False

while True:
    print_board()
    player_move('X')
    if is_victory('X'):
        print("X Wins! Congratulations!")
        break
    elif is_draw():
        print("It's a Draw!")
        break
    print_board()
    player_move('O')
    if is_victory('O'):
        print("O Wins! Congratulations!")
        break
    elif is_draw():
        print("It's a Draw!")
        break

```

## Output:

```

| |

```

```

-----

```

```

| |

```

```

-----

```

```

| |

```

Player 1's turn

Enter your move (1-9): 1

```

X| |

```

```

-----

```

```

| |

```

```

-----

```

```

| |

```

Player 2's turn

Enter your move (1-9): 3

```

X| |O

```

```

-----

```

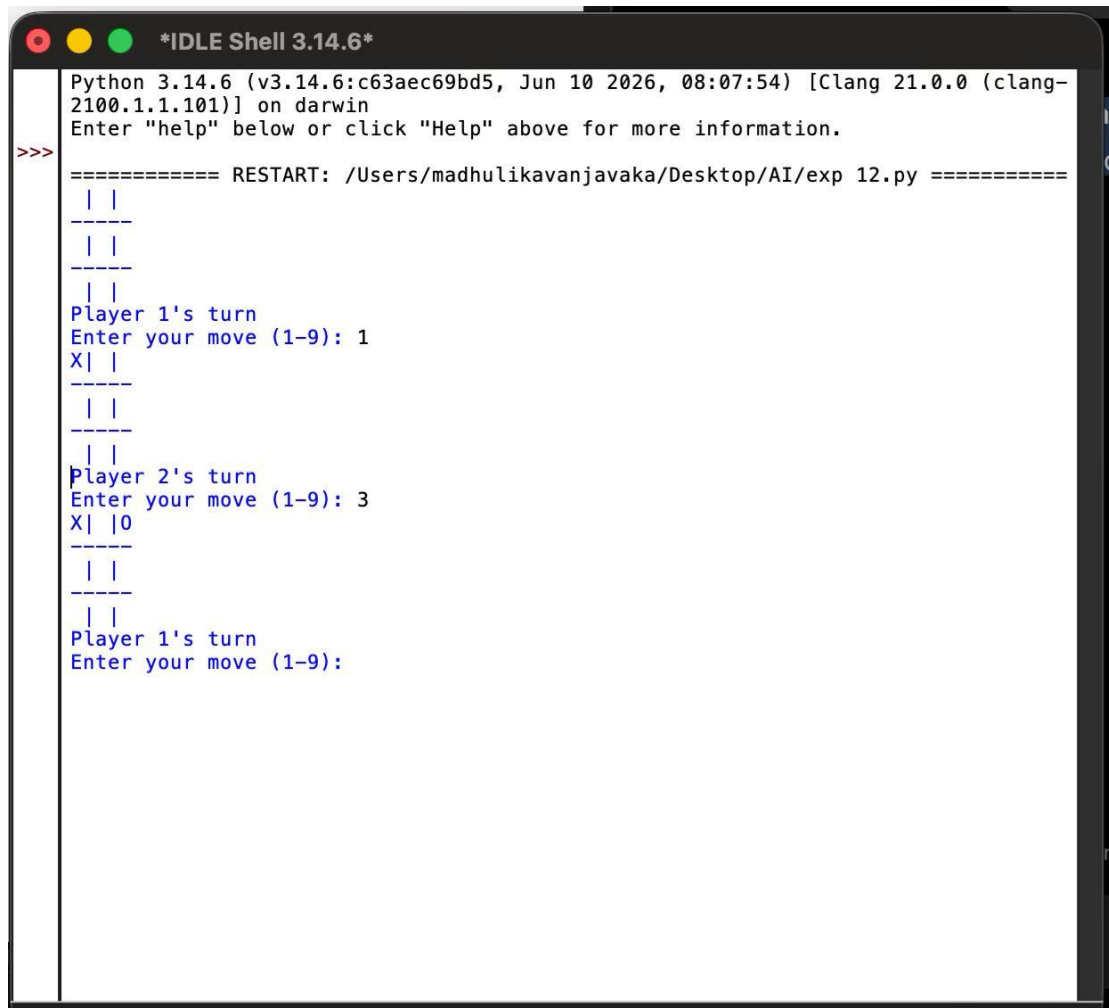
||

----

||

Player 1's turn

Enter your move (1-9):



```
*IDLE Shell 3.14.6*

Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/madhulikavanjavaka/Desktop/AI/exp 12.py =====
  ||
  ||
  ||
  ||
Player 1's turn
Enter your move (1-9): 1
X| |
  ||
  ||
  ||
Player 2's turn
Enter your move (1-9): 3
X| |O
  ||
  ||
  ||
Player 1's turn
Enter your move (1-9):
```

### Result:

Hence, the simple python program for Tic Tac Toe game is done.